IA TP Examen - IDS hybride pour la détection des intrusions réseau (SVM + K-Means)

IDS hybride pour la détection des intrusions réseau (SVM + K-Means)

1. Objectif du travail

Construire un projet applicatif Python qui analyse une connexion réseau décrite par le dataset NSL-KDD et prédit si elle est normale (0) ou intrusion (1), en combinant un classifieur SVM et un clustering K-Means, avec une interface graphique. Portée honnête : analyse d'un dataset en laboratoire uniquement, aucune capture ni détection temps réel sur un réseau réel.

2. Données utilisées

Pour l'entraînement de nos modèles nous avons utilisé le dataset NSL-KDD (fichiers KDDTrain+.txt et KDDTest+.txt fournis, format sans en-tête : 41 caractéristiques + étiquette + difficulté). Ce dataset contient 125 973 connexions d'entraînement : 67 343 normales (53,5 %) et 58 630 attaques réparties en 23 modalités (de neptune 41 214 à spy 2, perl 3). Vu que les attaques sont nombreuses et variées, nous les avons regroupées en une seule classe intrusion étiquetée 1, le trafic normal étant étiqueté 0. L'évaluation se fait sur KDDTest+ complet (22 544 connexions : 9 711 normales / 12 833 attaques), qui contient environ 17 types d'attaques inédites absentes de l'entraînement (mscan, apache2, saint, httptunnel...), ce qui mesure la généralisation.

3. Caractéristiques utilisées (features)

Chaque connexion est convertie en 117 nombres : 38 variables numériques (durée, octets source/destination, compteurs count/srv_count, taux d'erreur serror_rate..., indicateurs logged_in, root_shell...) standardisées (StandardScaler), plus 3 variables catégorielles (protocol_type, service, flag) transformées en 79 indicatrices (OneHot, services inédits du test ignorés). Nous sommes partis du principe qu'une intrusion se signale par des volumes, des compteurs ou des drapeaux anormaux (ex. count élevé, flag REJ répété).

4. Modèle SVM

Pour ça nous passons par 3 étapes que voici :

1. Standardisation des caractéristiques (point 3 de ce rapport), ajustée sur l'entraînement uniquement (pas de fuite de données).
2. Échantillon stratifié de 25 000 lignes de KDDTrain+ (seed 42, proportions normal/intrusion conservées), car un SVM RBF sur 125 973 lignes est trop coûteux pour un TP ; évaluation sur KDDTest+ complet.
3. Une connexion est classée par un SVM à noyau RBF (SVC, paramètres par

défaut de scikit-learn) : normale (0) ou intrusion (1).

5. Modèle K-Means

Avec ce modèle l'idée est de grouper les connexions par ressemblance sans utiliser les étiquettes 0 et 1 (non supervisé). K-Means (k=2, n_init=10) regroupe l'échantillon d'entraînement en 2 clusters ; chaque connexion est ensuite décrite par ses 2 distances aux centres.

6. Approche hybride

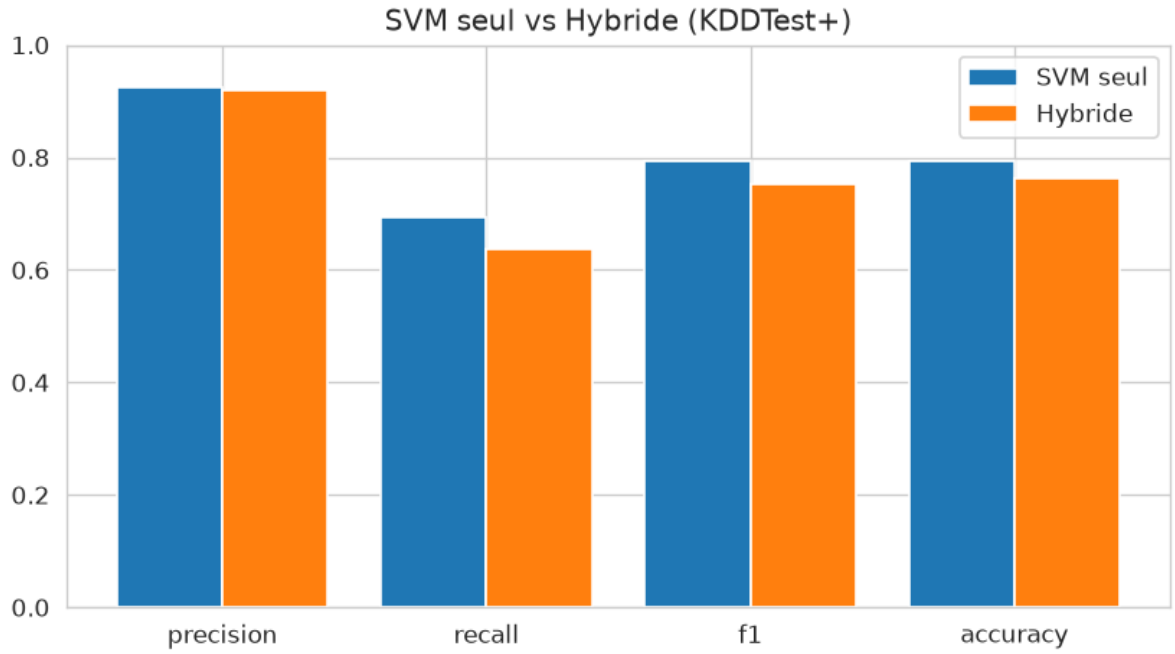
Les 2 distances aux centres K-Means sont ajoutées aux 117 caractéristiques, puis un second SVM (mêmes hyperparamètres RBF) est entraîné sur ces 119 valeurs. L'information de regroupement complète la frontière du classifieur. La comparaison est honnête : mêmes données d'entraînement, mêmes hyperparamètres et même jeu de test pour les deux modèles (seul le vecteur d'entrée change), comme dans le projet phishing-detector (K-NN + K-Means).

7. Résultats (KDDTest+ complet, 22 544 connexions)

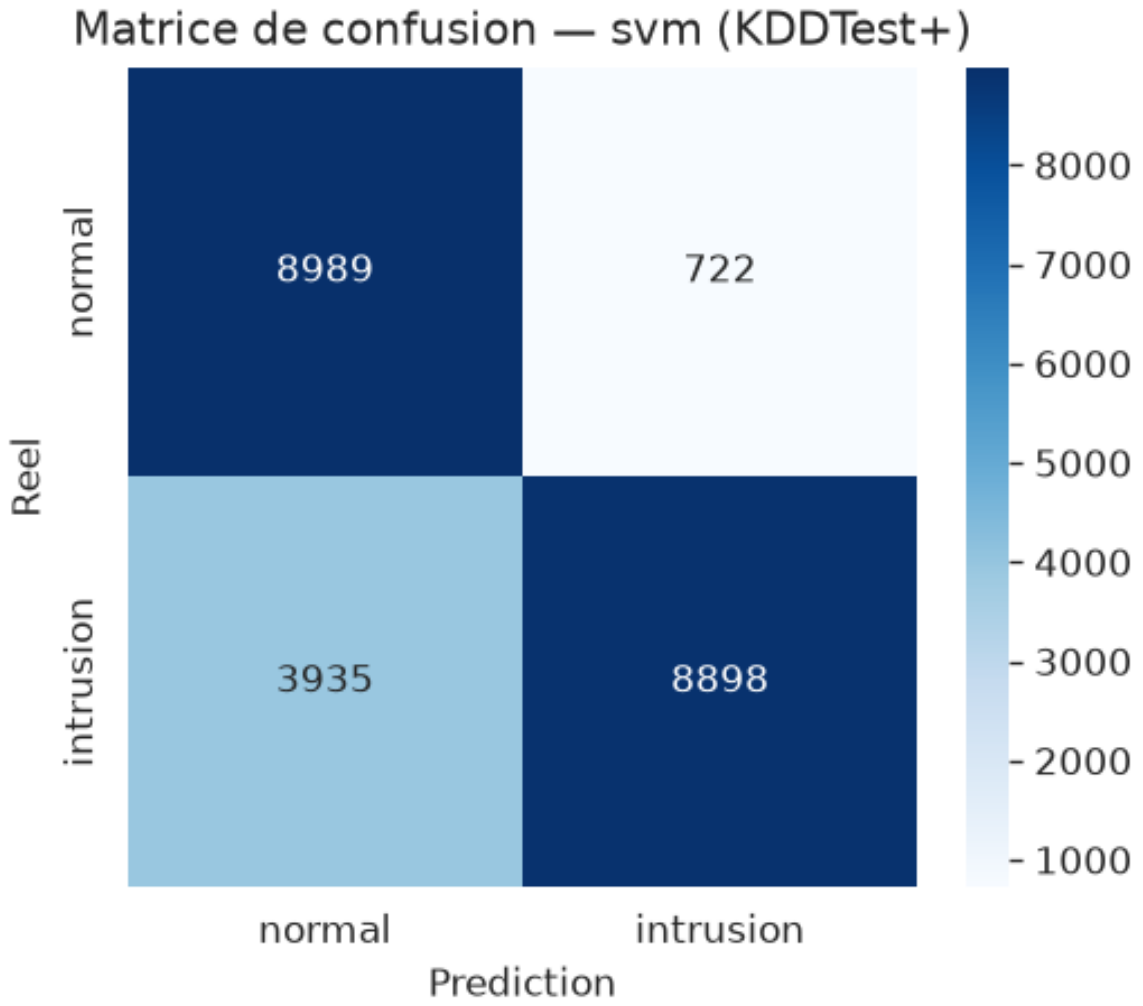
Modèle	Accuracy	Precision	Recall	F1	FPR
SVM seul	0.7934	0.9249	0.6934	0.7926	0.0743
Hybride	0.7622	0.9205	0.6372	0.7531	0.0727

Matrices de confusion [TN FP / FN TP] : SVM [[8989, 722], [3935, 8898]], hybride [[9005, 706], [4656, 8177]]. Variante K-Means k=5 testée : accuracy 0.7600, F1 0.7505 (pas mieux). Contrairement au projet phishing (gain modeste de l'hybride), ici l'hybride perd 3,1 points d'accuracy et 5,6 points de rappel : les attaques inédites du test tombent près du cluster normal et attirent la frontière du mauvais côté (721 intrusions manquées de plus). Le taux de faux positifs baisse légèrement (0,0743 -> 0,0727, 16 faux positifs évités). C'est un résultat honnête et défendable : le stacking aide quand les clusters séparent bien les classes, pas face à des familles inconnues. Graphiques comparatifs et matrices de confusion repris en images ci-dessous.

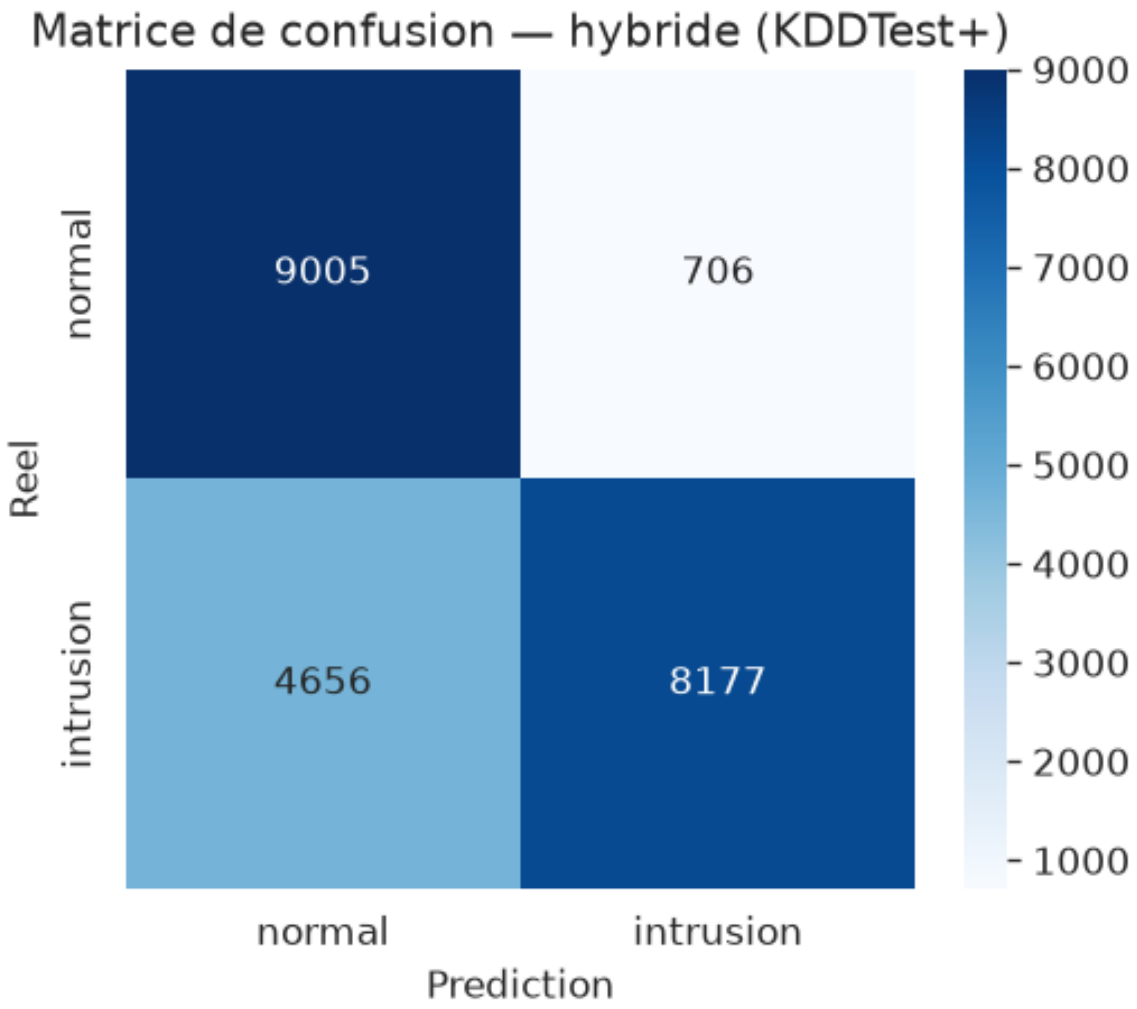
Comparaison des métriques (KDDTest+)



Matrice de confusion - SVM seul



Matrice de confusion - hybride



8. Optimisation

Pour ce projet nous l'avons optimisé en séparant l'entraînement des modèles, sauvegardés à part (scaler.pkl, encoder.pkl, svm.pkl, kmeans.pkl, svm_hybrid.pkl, meta.json), pour ne pas ralentir l'interface lors du démarrage de celle-ci : l'application recharge les modèles en quelques secondes (~50 s d'entraînement évitées à chaque lancement).

9. Interface

Pour ce projet nous avons utilisé la lib Streamlit pour faire plus rapidement des interfaces jolies, en 3 onglets : tester une connexion (formulaire de 10 champs clés, le reste = médianes de l'entraînement), analyser un fichier CSV, et voir les performances comparées.

Exemple avec une connexion normale (ligne réelle de KDDTest+)

tcp / ftp_data / SF, src_bytes 12983, count 1 -> SVM : Normal (0), hybride : Normal (0). Les deux modèles sont d'accord : connexion typique, classement correct.

Exemple avec une intrusion neptune (ligne réelle de KDDTest+)

tcp / private / REJ, src_bytes 0, count 229 -> SVM : Intrusion (1), hybride : Intrusion (1). Compteur très élevé et refus répétés : signature d'attaque par déni de service, classement correct.

Cas de divergence (attaque back réelle, pour le jury)

tcp / http / SF, src_bytes 54540, count 4 -> SVM : Intrusion (1, correct), hybride : Normal (0, manquée). 931 divergences de ce type sur 22 544 connexions : elles illustrent exactement la perte de rappel de l'hybride expliquée au point 7.